

Cuts and Negation in Prolog

Sujit Kumar Chakrabarti

International Institute of Information Technology Bangalore

1 The Cut Operator

2 Negation as Failure

1 The Cut Operator

2 Negation as Failure

Backtracking in Prolog

- Core feature of Prolog execution
- Explores alternative solutions automatically
- Can lead to:
 - Inefficiency
 - Unnecessary computation

Goal

Control backtracking when needed

The Cut Operator

- Written as !
- Arity: !/0
- Always succeeds ✓

✓ Effect

Commits Prolog to:

- Current clause ✓
- Choices made so far ✓

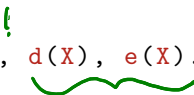
Example with Cut

```
p(X) :- b(X), c(X), !, d(X), e(X).
```

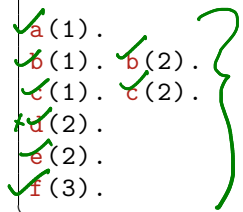
- Once ! is reached:
 - No alternative clauses of p/1
 - No alternative choices for b(X), c(X)

Cut-Free Program

```
p(X) :- a(X).  
p(X) :- b(X), c(X), d(X), e(X).  
p(X) :- f(X).
```



```
✓ a(1).  
✓ b(1). ✓ b(2).  
✓ c(1). ✓ c(2).  
✓ d(2).  
✓ e(2).  
✓ f(3).
```



■ Query: ?- p(X). gives: X = 1, 2, 3

Cut-Free Program

```
p(X) :- a(X).  
p(X) :- b(X), c(X), d(X), e(X).  
p(X) :- f(X).
```

```
a(1).  
b(1). b(2).  
c(1). c(2).  
d(2).  
e(2).  
f(3).
```

- Query: $?- p(X)$. gives: $X = 1, 2, 3$
- **Observation** Once the value of X is chosen for b and c , there's only one predicate left for d and e to explore.

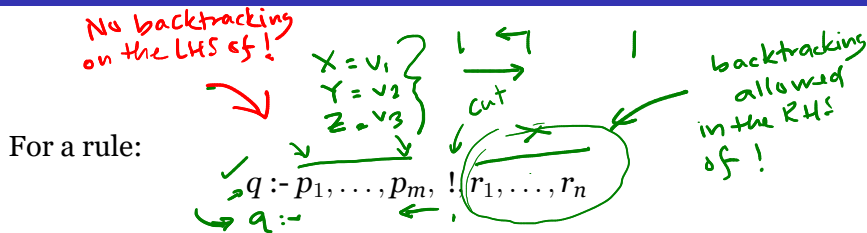
Adding a Cut

```
p(X) :- a(X).  
p(X) :- b(X), c(X), !, d(X), e(X).  
p(X) :- f(X).
```

Effect

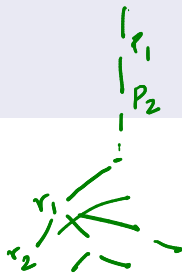
- After entering second clause, alternatives are pruned
- Some valid solutions may be lost

Meaning of Cut



Commitment

- Commit to this clause of q
- Commit to choices in $p_1, \dots, p_m$
- NOT to choices in $r_1, \dots, r_n$



Example: max/3

if $\text{arg3} = \text{max of arg1 and arg2}.$

→ $\text{max}(\text{X}, \text{Y}, \text{Y}) \text{ :- } \text{X} \leq \text{Y}.$
 $\text{max}(\text{X}, \text{Y}, \text{X}) \text{ :- } \text{X} > \text{Y}.$

- Works correctly
- But inefficient:
 - Tries second clause unnecessarily

max/3 with Cut

→ `max(X,Y,Y) :- X =< Y, !.`
→ `max(X,Y,X) :- X > Y.`

Advantage

- Avoids unnecessary backtracking
- Same logical meaning

Green Cuts

Definition

Cuts that do NOT change the meaning of a program

- Only improve efficiency
- Example: optimized `max/3`

Red Cuts

Definition

Cuts that change the meaning of a program

- Affect logical correctness
- Harder to understand
- Error-prone

Example of Red Cut

 $\text{max}(\textcolor{red}{X}, \textcolor{red}{Y}, \textcolor{red}{Y}) \text{ :- } \textcolor{red}{X} = < \textcolor{red}{Y}, !.$
 $\text{max}(\textcolor{red}{X}, \textcolor{red}{Y}, \textcolor{red}{X}).$

- Produces incorrect answers
- Meaning depends on cut

if $X=Y$ and rule 1
satisfies,
then rule 2 will be
missed.

1 The Cut Operator

2 Negation as Failure

The fail Predicate

- `fail/0` always fails
- Forces backtracking

Use

Combine with `cut` to encode exceptions

Cut + Fail Example

```
enjoys(vigyan,X) :-  
    setDosa(X), !, fail.  
enjoys(vigyan,X) :-  
    dosa(X).
```

- Excludes specific cases

Negation as Failure

Definition

```
neg(Goal) :- Goal, !, fail.  
neg(Goal).
```

- True if goal cannot be proven

Using Built-in Negation

- Prolog provides `\+`

```
enjoys(vigyan,X) :-  
    dosa(X),  
    \+ setDosa(X).
```

Important Note

Warning

Negation as failure is NOT logical negation

- Depends on proof failure
- Order of goals matters

Summary

- Cut controls backtracking
- Green cuts: safe optimization
- Red cuts: change meaning
- Negation as failure uses cut + fail